Control & Automation for the Efficient Use of Energy (CAPUEE 25–26)

Bujin Shao Hemeng Wang Nai-Chi Cheng Saúl Daniel Castañeda Arzaga

December 26, 2025

Abstract

Contents

1 Introduction

1.1 Background & Motivation

1.2 Problem Statement & Project Requirements

1.3 System Overview of the Proposed Solution

Figure 1: System schematic of the prototype.

2 Prototype Development — Hardware

2.1 Load of Choice

The *Xiaomi High-speed Ionic Hair Dryer* Model GSHGL01LX is used as the primary electrical load. With a rated power of 1600 W at a mains voltage of 220–240 V, it is suitable for validating our sensing and control system in a household context. It is also commonplace, as one of the team members already owned the unit, simplifying setup and data collection. Specifics:

- A high-speed internal motor is used for efficient airflow and drying, reflecting typical household power consumption patterns of motorized heaters.
- The nominal electrical characteristics—220–240 V at up to 1600 W—yield a current draw of approximately 7 A at rated conditions, which is a useful range for testing the robustness of our current transformer and ADC interface.
- Using a real shunt load like a hair dryer adds practical relevance to the project compared to purely resistive or artificial loads, and allows us to observe non-idealities such as inrush currents and thermal effects that occur during typical usage.
- The device includes a detachable nozzle, making it a relevant choice for evaluating non-invasive current sensing and thermal monitoring for the prototype by inserting the sensors between the nozzle and the main outlet.

Table 1: Specifications of the Hair Dryer (Xiaomi GSHGL01LX) [1].

Attribute	Value
Model	GSHGL01LX
Rated Voltage	220–240 V
Rated Frequency	50–60 Hz
Rated Power	1600 W
Product Dimensions	128.6 × 70 × 231 mm (without nozzle)
Power Cord Length	1.7 m
Net Weight	406 g (without nozzle and cord)

Figure 2: Visuals.

2.2 Electronics

Figure 3 on the next page shows a schematic of the electronics of the prototype. The design integrates the mains-powered hairdryer with low-voltage sensing and control electronics, while maintaining isolation between high-voltage and low-voltage domains.

Specifically, the hair dryer is connected to the AC mains, passing through an analog current transformer (SCT013) for real-time, non-invasive current measurement. The sensor output is conditioned, then digitized using an analog-to-digital converter (ADS1115), which communicates with a microcontroller (FireBeetle ESP32) via an I^2C bus. A temperature sensor module (LM35) is placed near the air outlet of the hair dryer, also connected to the microcontroller. A computer (not shown) handles control logic during testing to avoid reprogramming the microcontroller, which manages sensing and actuating. Control logic is not integrated for this prototype. All low-voltage components are powered from regulated DC supplies and share a common ground, while the AC side remains physically and electrically isolated for safety.

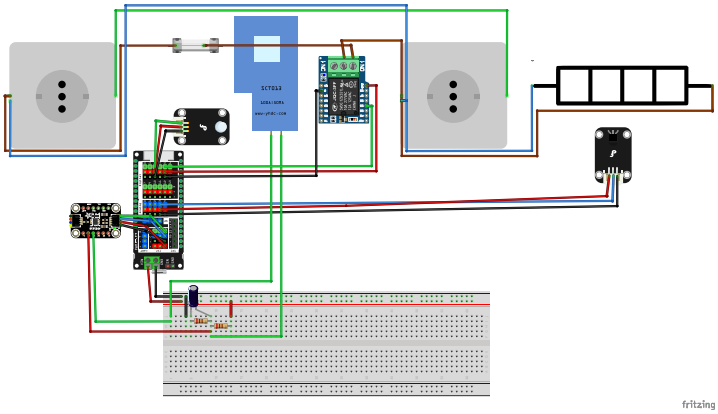


Figure 3: Hardware schematic of the prototype.

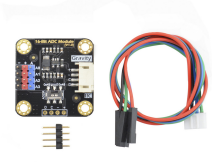
Component	Model	Purpose
Microcontroller	ESP32 FireBeetle	Control + connectivity
I/O expansion shield	Gravity	Wiring simplification
ADC module (16-bit)	ADS1115	High-resolution sampling
Current sensor	SCT013	Non-invasive AC current sensing
Temperature sensor module	LM35 (DFR0023)	Outlet air temperature
Digital relay module	HF3FA (DFR0473)	Actuator: power cutoff
LED module	DFR0021-B	Actuator: user feedback

Table 2: Bill of materials.

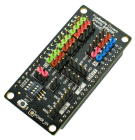
(a) SCT013 Current Sensor [2]

(b) Socket box with fuse and the relay in (c)

(c) HF3FA Relay [3]



(d) ADS1115 ADC [4]



(e) I/O Expansion Shield [5]

(f) FireBeetle ESP32 [6]

(g) LM35 Sensor [7]

(h) Blue LED Module [8]

Figure 4: Hardware components used in the experimental setup.

Current Measurement Chain (SCT013 + Bias + ADS1115)

The load current is measured using a non-invasive SCT013 current transformer, enabling galvanic isolation between the mains-powered appliance and the low-voltage measurement electronics. The sensor is clamped around the live conductor supplying the hair dryer, producing a secondary AC current proportional to the primary load current without requiring direct electrical contact.

The secondary output of the SCT013 is converted into a measurable voltage using a burden resistor, after which the signal is conditioned to interface with single-supply digital electronics. Since the sensor output is an AC waveform centered around 0 V, a midpoint bias network referenced to half of the 3.3 V supply is applied (1.65 V). This bias shifts the waveform into the valid input range of the analog-to-digital converter while preserving its amplitude and phase information. Basic passive filtering is included to reduce high-frequency noise and improve signal stability.

The conditioned and biased voltage signal is digitized using an external ADS1115 16-bit analog-to-digital converter. The ADS1115 is configured for single-ended input operation and samples the current measurement signal on channel AIN1 with respect to ground. Compared to the ESP32's internal ADC, the external ADC provides improved resolution, reduced noise sensitivity, and more repeatable measurements, which is particularly beneficial for low-frequency AC current monitoring.

All components of the current measurement chain operate in the low-voltage domain and share a common ground reference, while the SCT013 ensures electrical isolation from the high-voltage AC circuitry. This architecture allows safe and accurate acquisition of load current data suitable for further processing in software.

Temperature Measurement Chain (LM35)

The outlet air temperature is measured using an LM35 analog temperature sensor, which provides an output voltage linearly proportional to temperature (10 mV/°C). The sensor is powered from the 3.3 V supply and referenced to the common low-voltage ground, ensuring compatibility with the ESP32 analog input range.

The LM35 output is connected directly to an ESP32 ADC input pin, as the sensor's output voltage remains within safe limits for the microcontroller under the expected temperature range. Due to the analog nature of the signal, wiring between the sensor and the microcontroller was kept short to minimize noise pickup and ensure stable readings.

Physically, the sensor is positioned near the hair dryer air outlet and exposed to the outgoing airflow while avoiding direct contact with heating elements. This placement provides a representative measurement of operating temperature with acceptable response time, while reducing thermal stress and measurement instability caused by localized hot spots.

Basic validation of the temperature measurement was performed by comparing sensor output against ambient room temperature under no-load conditions and observing expected temperature rises during normal device operation. These checks confirmed correct sensor orientation, electrical connection, and thermal coupling prior to integration with the control logic.

Figure 5: Close-up schematic of the LM35 temperature measurement chain and ESP32 ADC connection.

Actuation Hardware (Blue LED / Relay Cutoff)

System actuation and user feedback are provided through a low-voltage relay and a blue status LED module, both controlled by the ESP32 via digital GPIO pins. The relay enables electrical isolation between the low-voltage control electronics and the mains-powered load, allowing the system to enable or disable the appliance safely.

The relay control input is driven by a dedicated GPIO pin configured as a digital output. The relay module incorporates the required driver and isolation circuitry to interface directly with 3.3 V logic levels, ensuring reliable switching while preventing high-voltage transients from propagating to the microcontroller. The relay contacts are located exclusively on the mains side of the circuit.

Visual system feedback is provided by a DFRobot digital blue LED module connected to a separate GPIO pin. The module integrates the LED and its current-limiting circuitry on-board, allowing it to be driven directly from a 3.3 V digital output without the need for an external series resistor. This simplifies wiring and reduces component count while maintaining safe operating conditions. The LED operates entirely within the low-voltage domain and provides a clear indication of system state (e.g., relay enabled or disabled).

Figure 6: Close-up schematic of the relay cutoff and blue LED module actuation circuitry controlled by the ESP32.

2.3 Hardware Build Photos

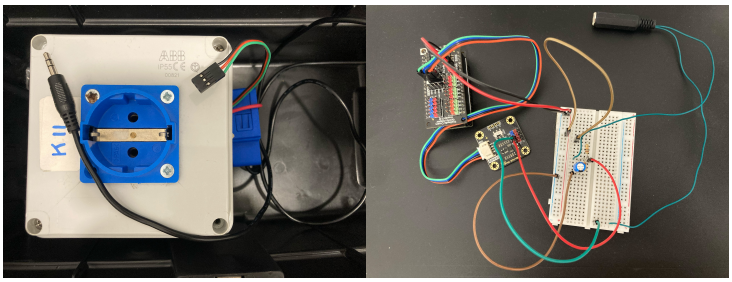


Figure 7: Hardware: AC side and DC side.

Figure 8: Hardware: full setup.

3 Prototype Development — Software

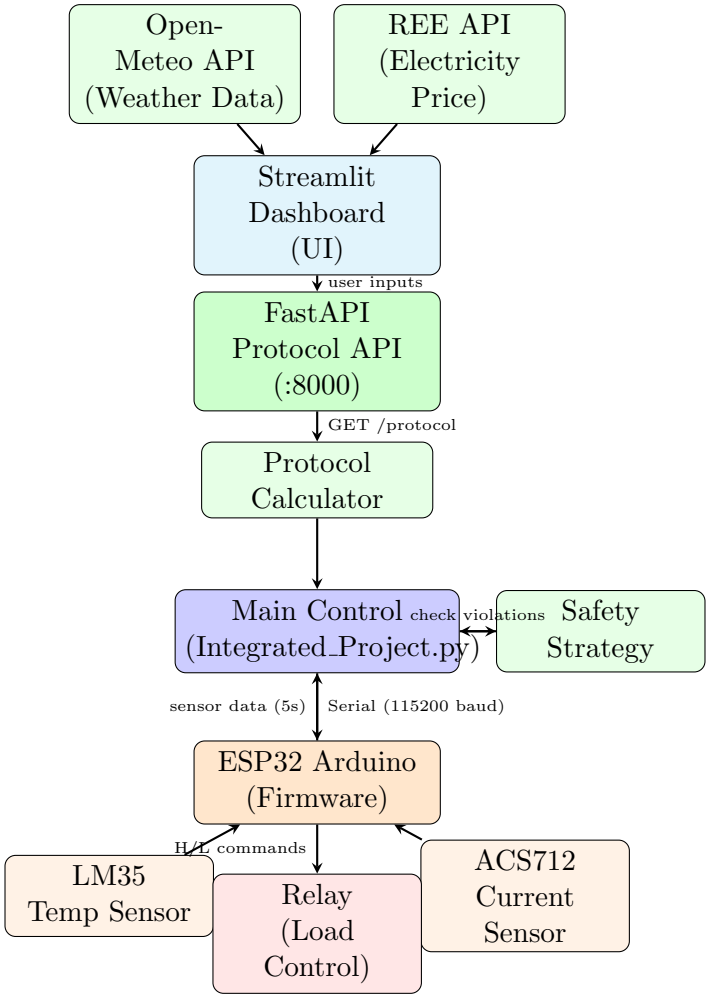


Figure 9: Software schematic of the prototype showing data flow between modules, external APIs, sensors, and actuators.

3.1 Software Architecture

The software system consists of six primary modules working in a distributed architecture:

ESP32 Arduino Firmware (`Integrated_Project.ino`) implements the lowest-level hardware interface, reading analog sensors through an ADS1115 16-bit ADC for current measurement and the ESP32's built-in ADC for temperature. The firmware samples current at 1 kHz to calculate RMS values over 20 ms intervals, then averages 250 RMS samples to produce one filtered output every 5 seconds. It communicates via serial at 115200 baud, responding to relay control commands ('H' for high/on, 'L' for low/off) and transmitting sensor readings in the format: `Temperature: XX.XX | Current: X.XXXXX`.

Main Control System (`Integrated_Project.py`) serves as the central coordinator, managing three key responsibilities: (1) serial communication with the ESP32 through the `SerialBoard` class, (2) protocol acquisition from the local API with user-specific parameters (hair type, porosity, environmental conditions), and (3) real-time safety monitoring by feeding sensor data to the `SafetyStrategy` module. It runs a continuous monitoring loop synchronized to the Arduino's 5-second output interval, checking for threshold violations and commanding relay shutdown when safety limits are exceeded.

Protocol API (`mock_protocol_api.py`) provides a RESTful interface via FastAPI at `http://127.0.0.1:8`

accepting query parameters including room temperature, humidity, hair type, porosity, towel-dry level, time priority, and safety thresholds. It delegates calculation logic to the protocol calculator module and returns a JSON response specifying sensor-specific thresholds (temperature and current) with corresponding duration limits.

Protocol Calculator (`protocol_calculator.py`) implements the core recommendation engine, calculating optimal drying parameters based on environmental factors and user characteristics. It computes a drying power index from temperature and humidity, determines appropriate heat settings (Low/Medium/High), estimates bulk and finish phase durations, and generates safety protocol thresholds. It also interfaces with external APIs to fetch real-time room conditions from Open-Meteo and electricity prices from the Spanish REE API or regional fallback values.

Safety Strategy Module (`safety_strategy.py`) monitors sensor readings against protocol-defined thresholds with duration-based violation detection. For each safety condition (temperature and current), it starts a timer when a threshold is exceeded and triggers a shutdown if the condition persists beyond the specified duration. It implements hysteresis by resetting timers when values return below thresholds, preventing false triggers from transient spikes.

Streamlit Dashboard (`StreamlitDashboard.py`) provides a web-based user interface for inputting location, hair characteristics (type and porosity), and viewing personalized drying protocols. It displays environmental conditions, recommended drying phases (towel-dry, optional air-dry, bulk heat, and finish), total time estimates, and energy cost analysis across multiple time-priority and towel-dry level combinations. The dashboard visualizes cost trade-offs through interactive charts, helping users select optimal drying strategies.

3.2 Data Streams and Sources

The system integrates multiple external and internal data sources with specific update frequencies and failure handling:

Open-Meteo Weather API provides current temperature and relative humidity for user-specified locations through two endpoints: (1) geocoding API (`https://geocoding-api.open-meteo.com/v1/s`) converts city names to coordinates, and (2) forecast API (`https://api.open-meteo.com/v1/forecast`) returns current weather with `temperature_2m` and `relative_humidity_2m` fields. Requests timeout after 5 seconds, falling back to Barcelona coordinates (41.3874°N, 2.1686°E) or default values (20°C, 60% humidity) on failure.

REE Electricity Price API delivers Spanish real-time electricity prices from `https://apidatos.ree.es/` with hourly granularity. The system queries today's data (midnight to midnight UTC), extracts the current hour's price in €/MWh, and converts to €/kWh. For non-Spanish locations or API failures, it uses a regional price map with fallback values ranging from 0.14 (Canada) to 0.35 (Germany) €/kWh.

Arduino Sensor Data Stream transmits temperature and current readings every 5 seconds over serial. Temperature measurements use the LM35's linear 10 mV/°C characteristic: $T = V \times 100$, where V is the ADC voltage. Current measurements undergo two-stage filtering: RMS calculation over 20 ms windows (matching AC power frequency), then averaging 250 RMS samples for noise reduction. The 5-second interval balances responsiveness with computational efficiency, as the ESP32 performs $250 \times 20 = 5000$ ms of RMS integration per output.

Serial Communication Protocol uses a simple ASCII format for bidirectional communication at 115200 baud. Commands from Python to Arduino consist of single characters ('H' for relay on, 'L' for relay off, 'PING' for handshake verification). Sensor data from Arduino follows the template `Temperature: XX.XX | Current: X.XXXXX`, parsed by splitting on the pipe character and extracting float values after colons. The Python `SerialBoard` class buffers incoming data in a background thread, updating `latest_temp` and `latest_current` attributes that the main control loop reads.

3.3 Control and Recommendation Logic

The system implements three-tier safety thresholds with duration-based triggering and hysteresis:

Temperature Threshold Monitoring: The protocol specifies a maximum temperature (e.g., 38°C measured at sensor location) and total duration limit based on calculated drying time. When sensor temperature exceeds threshold, `SafetyStrategy` records the violation timestamp. If temperature remains elevated for the full duration (e.g., total drying time of 4.2 minutes = 252 seconds in demo mode), the system triggers shutdown with the message "You used too long the total hairdrying duration." If temperature drops below threshold before duration expires, the timer resets, implementing hysteresis to prevent oscillation.

Current Threshold Monitoring: High current indicates high heat mode usage on the hairdryer. The system monitors for sustained current above threshold (e.g., 5.0 A) for the bulk phase duration (e.g., 2.4 minutes). This prevents prolonged high-heat exposure that could damage hair, triggering shutdown with "You used too long the high heat mode in your hairdryer" when violated. The duration corresponds to the bulk drying phase, allowing sufficient time for the high-heat portion while preventing overuse.

Maximum Runtime Limit: A failsafe 30-minute (1,800,000 ms) absolute runtime limit prevents indefinite operation regardless of sensor readings. This protects against sensor failures or unexpected usage patterns. In demo mode, all durations divide by 10 for faster testing (e.g., 30 minutes becomes 3 minutes).

Hysteresis Implementation: The `SafetyStrategy.check_violations()` method maintains per-condition state including `triggered_at` timestamps. Violations only count when continuously exceeding thresholds for full durations. Dropping below threshold at any point resets the timer to `None`, requiring a fresh sustained violation to trigger shutdown. This design prevents false alarms from brief temperature spikes or momentary high-heat bursts during normal hair manipulation.

Recommendation Engine: The protocol calculator determines heat settings through a multi-factor index. Starting with a hair-type baseline (fine=0, medium=1, thick=2), it adjusts for porosity (low +1, high -1), environmental drying power (DP $<$ 0.6 adds +1, DP $>$ 1.2 subtracts -1), and time priority (scaled by $0.5 \times (\text{priority} - 0.5)$). The final heat index, clamped to 0-2, maps to Low (50-60°C), Medium (70-80°C), or High (90-100°C) settings.

Drying time calculation begins with a base bulk time:

$$t_{\text{bulk}} = \left(2 + \frac{\text{towel\_level} - 50}{25}\right) \times k_{\text{hair}} \div (0.5 + 0.5 \times \text{DP}) \div (1 + 0.3 \times \text{priority}) \tag{1}$$

where $k_{\text{hair}} = \{0.7, 1.0, 1.4\}$ for fine/medium/thick hair. The finish time follows a similar formula with different coefficients. For favorable conditions (temp $\geq$ 18°C, humidity $<$ 75%, DP $\geq$ 0.5, priority $\leq$ 0.7), the system recommends hybrid drying: 10-30 minutes of air drying followed by reduced blow-dry times (bulk $\times$0.6, finish $\times$0.7).

3.4 Details of Key Algorithms and Communications

RMS Current Calculation: The firmware computes true RMS using numerical integration synchronized to the AC power frequency (50 Hz). For each 1 ms timestep:

$$S_{\text{RMS}} = S_{\text{RMS}} + i^2(t) \cdot \Delta t \tag{2}$$

where $i(t) = (V_{\text{ADC}} - 1.65) \times 30$ converts the ACS712's AC-coupled voltage to current, and Δt in seconds is tracked by the microcontroller. After accumulating 20 samples (20 ms = one AC period at 50 Hz):

$$I_{\text{RMS}} = \sqrt{f \cdot S_{\text{RMS}}} \tag{3}$$

where $f = 50$ Hz. The factor of frequency converts the time-integrated sum to proper RMS. This method accurately captures non-sinusoidal waveforms from variable-speed hairdryer motors.

Multi-Stage Filtering: Raw RMS values undergo moving average filtering over 250 samples to reject high-frequency noise and short-term variations:

$$I_{\text{filtered}} = \frac{1}{250} \sum_{k=0}^{249} I_{\text{RMS}}[k] \tag{4}$$

This 5-second averaging window (250 samples $\times$ 20 ms) provides stable readings while remaining responsive to actual usage changes. Values below 0.1 A are clamped to zero to eliminate sensor offset noise.

Serial Communication Handshake: On connection, the Python client sends `PING\textbackslashr\tex` and waits 500 ms for a response containing "OK" or "PONG". If no response arrives but the serial port opened successfully, the system assumes a valid Arduino connection and proceeds. This graceful degradation allows operation even if the Arduino firmware doesn't implement PING handling, while providing verification when available.

Automatic Port Discovery: The `SerialBoard` class iterates through all available COM/tty ports, attempting connection to each until successful. For each port, it opens the serial connection, waits 2 seconds for ESP32 initialization (required for USB-serial enumeration), attempts handshake, and moves to the next port on failure. This eliminates manual port configuration.

Threaded Sensor Reading: A daemon thread runs `read_sensor_data()` continuously, parsing incoming serial lines and updating `latest_temp` and `latest_current` attributes. The main control loop reads these attributes every 5 seconds, decoupling sensor I/O from safety logic. Thread synchronization relies on Python's GIL, as float assignments are atomic operations.

Demo Mode Acceleration: Setting `DEMO_MODE = 1` divides all protocol durations by 10 before sending to `SafetyStrategy`, allowing rapid testing (e.g., 4-minute protocols become 24 seconds). Critically, this only affects safety timeout durations, not the Arduino's sensor sampling or output rate, maintaining realistic sensor data flow during accelerated testing.

Figure 10: Example UI/logging screenshot.

4 Testing and Validation Iterations

4.1 Test Plan

4.2 Calibration Iterations

4.3 Failure Modes & Mitigations

5 Results and Conclusion

5.1 Performance

Current and Power Measurement Results

Temperature Profiles

Control Outcomes

Energy/Cost Impact Estimation

Limitations and Improvements

5.2 Discussion

Summary of Accomplishments

Key Takeaways

References

[1] Xiaomi, *Xiaomi High-speed Ionic Hair Dryer: Technical Specifications*. Xiaomi, 2023. Online product specification page.

[2] YHDC, *SCT-013 Non-Invasive AC Current Transformer*. YHDC, 2018. Datasheet.

[3] DFRobot, *Gravity: Digital Relay Module (Arduino & Raspberry Pi Compatible) (SKU: DFR0473)*. DFRobot, 2023. Online hardware specification and user documentation.

[4] Texas Instruments, *ADS1115 16-Bit Analog-to-Digital Converter with I^2C Interface*. Texas Instruments, 2015. Datasheet.

[5] DFRobot, *Gravity Expansion Shield for FireBeetle 2*. DFRobot, 2023. Online hardware specification and pinout documentation.

[6] DFRobot, *FireBeetle ESP32 IoT Microcontroller (V3.0)*. DFRobot, 2023. Online hardware specification and user documentation.

[7] National Semiconductor, *LM35 Precision Centigrade Temperature Sensors*. National Semiconductor, 2000. Datasheet.

[8] DFRobot, *Digital White LED Light Module (SKU: DFR0021)*. DFRobot, 2023. Online hardware specification and user documentation.